

Handling Missing values

Jayita Chakraborty (PhD)

Handling Missing Data

Missing data in time-series analysis is quite prevalent. In certain cases, you may have to think whether the data is actually missing. Before filling in the missing value with mean or zero, we have to consider the following aspects:

- Let's say you have travelling distance to office of various participants over time. Now, some days you observed that the values are zero. Before arriving at the conclusion, you need to ask were these holidays, or did the person work from home at that day.
- In our previous session, we looked at the stock price dataset. If you have observed that stock price dataset did not have data for Saturday and Sunday. This is normal. But if you fill them with zeros, the model looking at the immediate past to predict the future will generate inaccurate values, especially for Monday.
- There can also be instances of data inaccuracies due to incorrect data entry or malfunction of the equipment involved. For example, GPS data, traffic sensor data.

Missing Data Mechanisms

Depending on what causes missing data, the gaps will have a certain distribution. Missing data mechanisms can be divided into three categories “*Missing completely at random*” (MCAR), “*Missing at random*” (MAR) and “*Not missing at random*” (NMAR). In practice, assigning data-gaps to a category can be blurry, because the underlying mechanisms are simply unknown.

- **Missing completely at random (MCAR)**

In MCAR there is no systematic mechanism on the way the data is missing. Missing data points occur entirely at random. This means there are two requirements: First, the probability for a certain observation being missing is independent from the values of other variables. Second, the probability for an observation being missing is also independent of the value of the observation itself.

Example: Sensor data is recorded 24/7 from a field test. Due to unknown reasons and on random occasions sometimes the transmission fails.

- **Missing at random (MAR)**

Like in MCAR, in MAR the probability for an observation being missing is also independent of the value of the observation itself. But it is dependent on other variables.

Example: Stock Price data are more likely to be missing on weekends (since it is shut down on some weekends).

- **Not missing at random (NMAR)**

NMAR observations are not missing in a random manner. The data are not MCAR and not MAR. That means, the probability for a observation being missing depends on the value of the observation itself. Furthermore the probability can (but must not necessarily) be dependent on other variables.

Example: Temperature sensor gives no values for temperatures over 100 degrees.

```
require(tidyverse)
require(xts)
```

Let's look at an Air Quality [dataset](#) published by the ACT Government, Canberra, Australia, under the CC by Attribution 4.0 International License and see how we can impute such values.

```
df <- read.csv("./data/Air_Quality_Monitoring_Data.csv")

str(df)
```

```
## 'data.frame':    15246 obs. of  22 variables:
## $ Name       : chr  "Monash" "Florey" "Civic" "Civic" ...
## $ GPS        : chr  "(-35.418302, 149.094018)" "(-35.220606, 149.0435)" "(-35.285399, 149.131536)" "(-35.285399, 149.131536)" ...
## $ DateTime   : chr  "31/07/2022 11:00:00 PM" "31/07/2022 11:00:00 PM" "31/07/2022 11:00:00 PM" "31/07/2022 11:00:00 PM" ...
## $ NO2        : num  0.007 0.009 NA NA 0.009 0.011 NA 0.013 0.016 NA ...
## $ O3_1hr     : num  0.016 0.001 0.004 0.007 0.014 0 0.008 0 0.005 0.008 ...
## $ O3_4hr     : num  0.011 0 0.007 0.008 0.01 0.001 0.01 0.002 0.012 0.013 ...
## $ O3_8hr     : num  0.014 0.003 0.01 0.012 0.015 0.006 0.014 0.008 0.016 0.016 ...
## $ CO         : num  0.19 0.57 NA NA 0.18 0.48 NA 0.4 0.15 NA ...
## $ PM10.1.hr  : num  11.3 21.4 14.7 15.3 14.3 16.2 16.9 11.9 18.5 10.8 ...
## $ PM2.5.1.hr : num  11.5 24 11.1 11.7 12.7 21.4 10.8 29.2 11.8 11.1 ...
## $ PM10       : num  26.9 26.6 9.9 10.2 28.6 27.7 10.3 28.6 29.8 10.5 ...
## $ PM2.5      : num  24.5 24.4 9 9 25.9 25.4 9 26.4 27 9 ...
## $ AQI_CO     : int  2 6 NA NA 2 5 NA 4 2 NA ...
## $ AQI_NO2    : int  9 11 NA NA 11 14 NA 16 20 NA ...
## $ AQI_O3_1hr : int  16 1 4 7 14 0 8 0 5 8 ...
## $ AQI_O3_4hr : int  13 0 8 10 13 1 13 3 14 16 ...
## $ AQI_O3_8hr : int  22 NA 15 19 23 NA 22 NA 25 25 ...
## $ AQI_PM10   : int  54 53 20 20 57 55 21 57 60 21 ...
## $ AQI_PM2.5  : int  98 98 36 36 103 101 36 105 108 36 ...
## $ AQI_Site   : int  98 98 36 36 103 101 36 105 108 36 ...
## $ Date       : chr  "31 July 2022" "31 July 2022" "31 July 2022" "31 July 2022" ...
## $ Time       : chr  "23:00" "23:00" "23:00" "22:00" ...
```

We have chosen region **Monash** and **PM2.5** readings.

```
df_filter <- df %>%
  filter(Name == "Monash") %>%
  select(DateTime, PM2.5)
```

Now, let's explore the dataset. Before we look out for any missing data, first we need to ensure that R recognises our **time** variable and assigns the correct data type. Now, look at the time variable in the dataset. Is it having the correct data type? Which function from **lubridate** is going to be applicable here?

Once, we have parsed our time variable correctly, we can extract more information. Let's us extract the **date**, **hour** and **time**.

```
df_filter <- df_filter %>%
  mutate(date = date(DateTime), hour = hour(DateTime), time = hms::as_hms(DateTime))
```

Now, we fill filter our dataset for a specific time interval and introduce some artificial missing values in the data. We are going to assign NA values randomly.

The first step to any analysis should be to inspect and visualize your data. Dealing with NAs is no different. If we know there are NAs in our time series we need to see when they occur and how often. Here I've used **{ggplot2}** to make a custom plot to visualise the occurrence of NA values at the scale of days.

```
df_filter_na <- df_filter %>%
  mutate(PM2.5_missing = PM2.5) %>%
  filter(DateTime >= as.POSIXct("2022-01-01 00:00:00"), DateTime <=
    as.POSIXct("2022-04-30 23:59:59"))

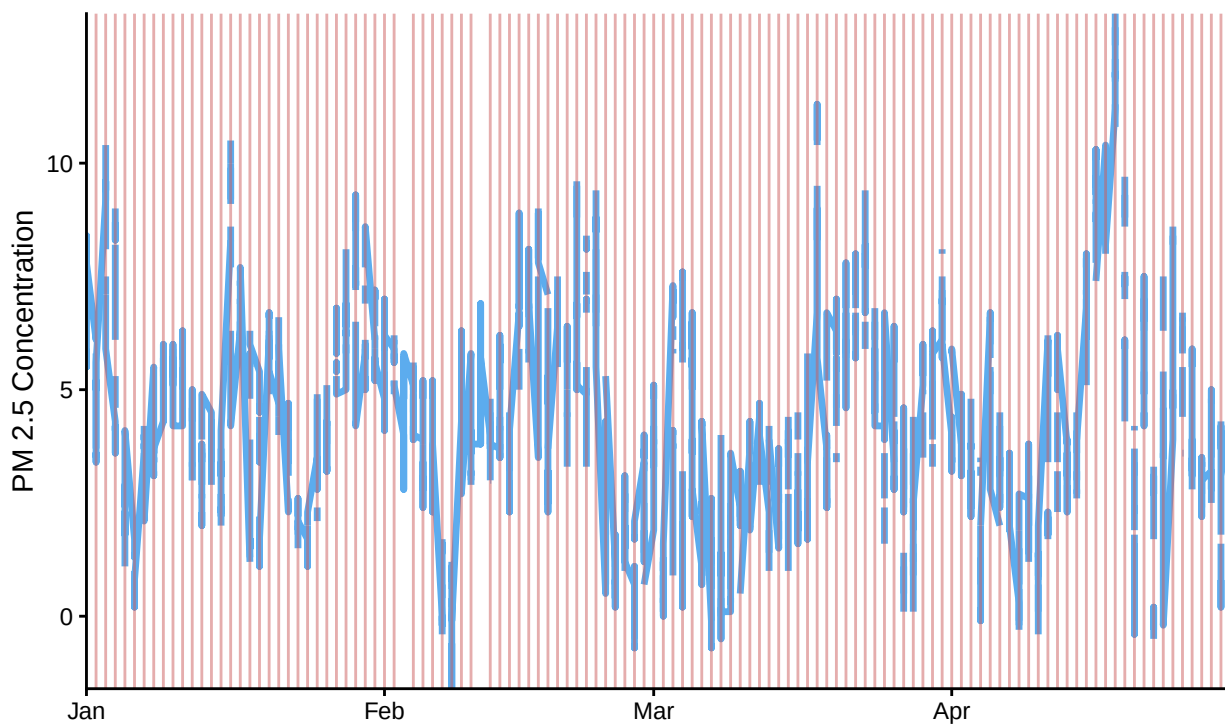
## Set seed to make it repeatable
set.seed(1234)

df_filter_na$PM2.5_missing[sample(1:(nrow(df_filter_na) - 1),
  size = 500)] <- NA

df_filter_na %>%
  {
    ggplot() + geom_line(data = ., aes(x = date, y = PM2.5_missing),
      colour = "steelblue2", linewidth = 1.2) + geom_vline(data = filter(.,
        is.na(PM2.5_missing)), aes(xintercept = date), colour = "indianred",
        alpha = 0.5) + scale_x_date(expand = c(0, 0)) + scale_y_continuous(expand = c(0,
        0)) + labs(title = "Location of missing values",
        subtitle = "Time series with highlighted missing regions",
        y = "PM 2.5 Concentration", x = "") + theme_classic()
  }
```

Location of missing values

Time series with highlighted missing regions

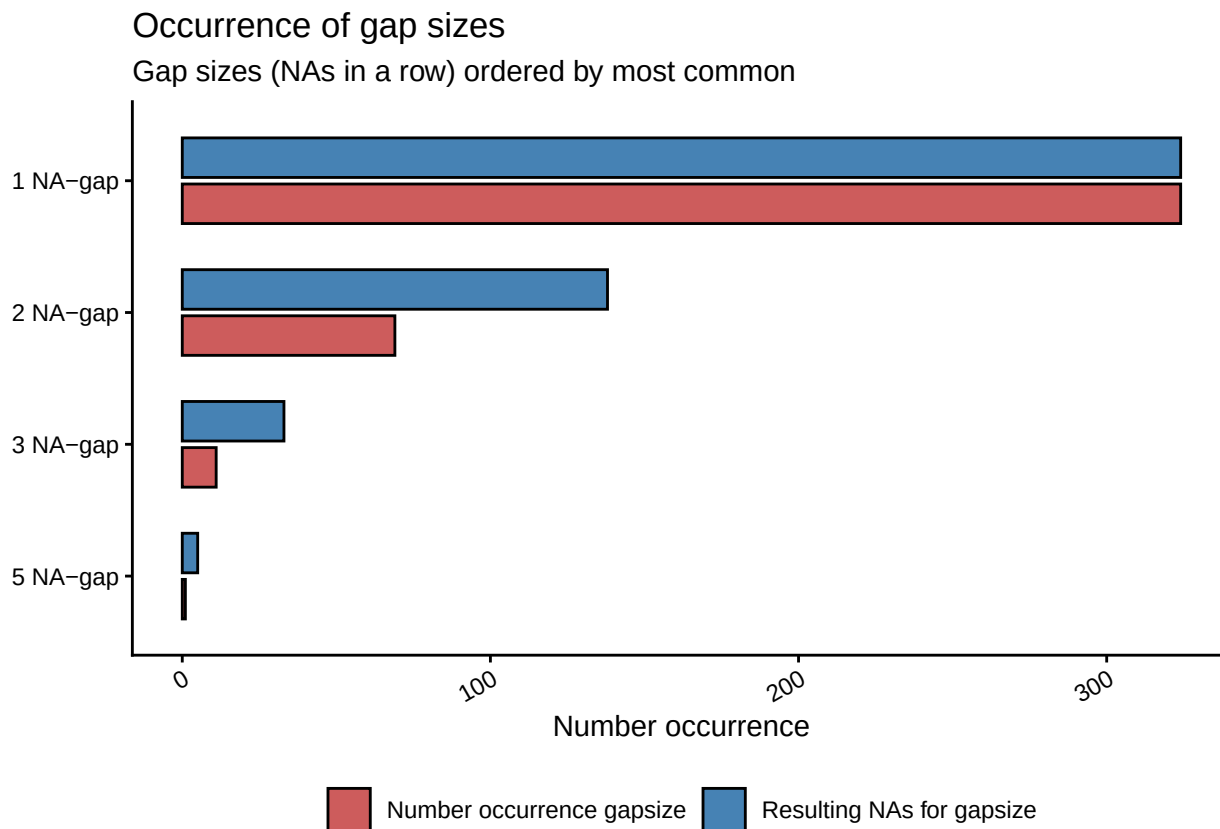


The

`imputeTS` package offers implementations of various time series imputation algorithms for missing data statistics. It also provides certain visualisation functions to investigate missing data in time-series analysis. You can read more about the package [here](#). The inbuilt function `ggplot_na_gapsize()` from `{imputeTS}` is handy to see how often NAs occur consecutively. We can see that most NAs occur alone.

```
require(imputeTS)

ggplot_na_gapsizes(df_filter_na$PM2.5_missing, ranked_by = "total",
  theme = ggplot2::theme_classic())
```



In our previous class, we discussed about time-series objects. In R, there are four different time-series objects that you will come across: `ts` from base R, `zoo`, `xts` and `tsibble` from packages. Now, `ts` will always expect a regular time-series and cannot handle irregular gaps in your data. `zoo`/`xts` is capable of handling irregular time series. Compared to the conventional time series objects, the `tsibble` preserves time indices as the essential data column and makes heterogeneous data structures possible. You can automatically convert your `ts`/`zoo`/`xts` objects to `tsibble`. If it is a tibble or data frame, you will need to declare the index and key variables. Now, it can sometimes become confusing to choose among the available options. If you have a clean univariate regular time-series data and want to conduct some simple forecasting, you can use `ts`. If you have clean, multivariate, irregular/regular time-series data, you can use `zoo`/`xts`. If you are at the *initial stage of your project*, start with `tsibble`. According to me, it is not advisable to convert your data to a time-series object before you are completely sure about the pre-processing. The following figure 1 provides a brief workflow on determining the appropriate class for your time-series data. Also look at this [blog](#) by Selva Prabhakaran for further insights.

```
require(tsibble)

df_filter_tibble <- as_tsibble(df_filter_na, index = DateTime)
```

Imputation Techniques

Now, let's look at a few simple techniques we can use to fill in the missing values:

- **Last Observation Carried Forward or Forward Fill:** This imputation technique takes the last observed

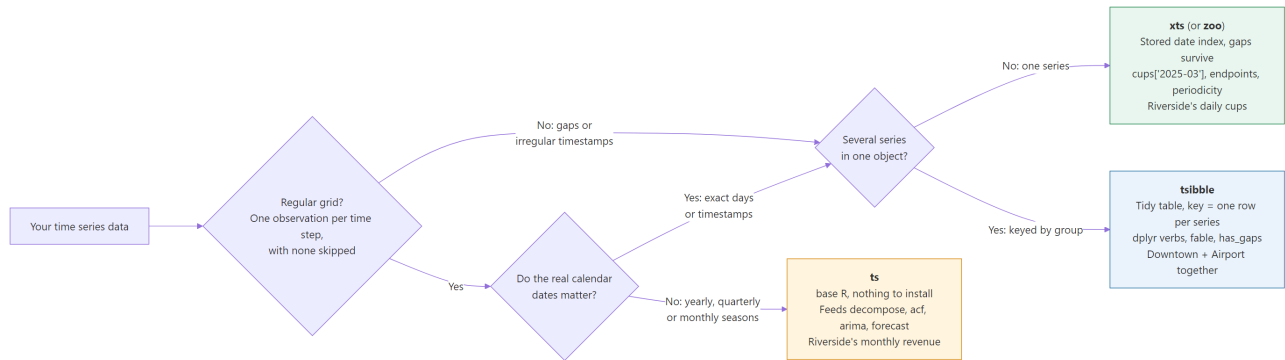


Figure 1: Workflow for picking the class

value, using that to fill in all the missing values until it finds the next observation. This is also called *forward fill*.

- **Next Observation Carried Backward or Backward Fill:** This imputation technique takes the next observation and backtracks to fill in all the missing values with this value. This is also called *backward fill*.

We can use this technique in R through the `na_locf` function in the `{imputeTS}` package.

```
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_ffill = na_locf(PM2.5_missing), PM2.5_bfill = na_locf(PM2.5_missing,
    option = "nocb"))
```

We will also calculate the mean and standard error from our real data to compare the techniques later.

```
## The mean and SE of PM 2.5 from the real data
fullmean <- mean(df_filter_tibble$PM2.5)
fullSE <- sd(df_filter_tibble$PM2.5)/sqrt(nrow(df_filter_tibble))

print(paste("Full sample: Mean: ", round(fullmean, 4), " SE:",
  round(fullSE, 4)))
```

```
## [1] "Full sample: Mean:  4.2835  SE: 0.0444"
```

Let's check how the mean and standard error from these two techniques compare to our real data.

```
## The mean and SE of PM 2.5 with ffill and bfill
ffillmean <- mean(df_filter_tibble$PM2.5_ffill)
ffillSE <- sd(df_filter_tibble$PM2.5_ffill)/sqrt(nrow(df_filter_tibble))

bfillmean <- mean(df_filter_tibble$PM2.5_bfill)
bfillSE <- sd(df_filter_tibble$PM2.5_bfill)/sqrt(nrow(df_filter_tibble))

cat("With Forward Fill: Mean: ", round(ffillmean, 4), "SE:",
  round(ffillSE, 4), "\n", "With Backward Fill: Mean: ", round(bfillmean,
  4), " SE:", round(bfillSE, 4), "\n")
```

```
## With Forward Fill: Mean:  4.2879 SE: 0.0444
## With Backward Fill: Mean:  4.2765 SE: 0.0443
```

The difference is not much. We will try out some more techniques.

- **Global Mean Substitution** : The simplest approach that is sometimes proposed is to replace missing data with a global mean.

```
df_filter_tibble <- df_filter_tibble %>%  
  mutate(PM2.5_meansub = case_when(is.na(PM2.5_missing) ~ mean(PM2.5_missing,  
    na.rm = TRUE), TRUE ~ PM2.5_missing))
```

Seems easy enough, but this raises a host of serious issues. Firstly, by replacing NAs with the global mean we artificially reduce the uncertainty when we try to estimate population statistics.

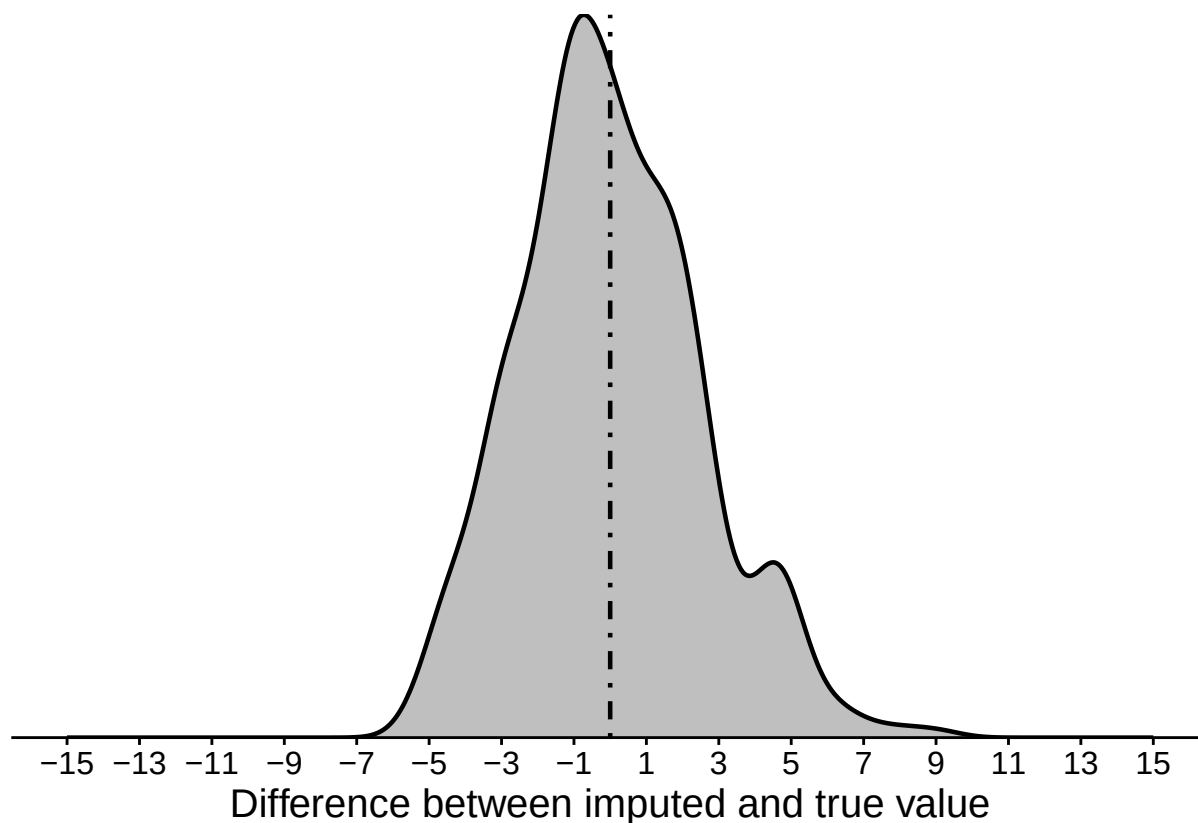
```
## The mean and SE of PM 2.5 conc data with global mean  
## substitution SE is (incorrectly) reduced.  
imputedmean <- mean(df_filter_tibble$PM2.5_meansub)  
imputedSE <- sd(df_filter_tibble$PM2.5_meansub)/sqrt(nrow(df_filter_tibble))  
  
print(paste("With global mean substitution: Mean: ", round(imputedmean,  
  4), " SE:", round(imputedSE, 4)))
```

```
## [1] "With global mean substitution: Mean:  4.2804  SE: 0.0399"
```

In the example above, we can see that the standard error of our PM 2.5 concentration estimate is (incorrectly) reduced when we replace NAs with the global mean.

We will also create bias if we want to use our variable as a response in statistical analysis, like linear regression. We will tend to under-estimate the relationship between our predictor and response variable.

```
filter(df_filter_tibble, is.na(PM2.5_missing)) %>%  
  mutate(diff = PM2.5 - PM2.5_meansub) %>%  
  {  
    ggplot(.) + geom_density(aes(x = diff), colour = "black",  
      fill = "grey75", linewidth = 0.8) + geom_vline(xintercept = 0,  
      lty = 10, lwd = 0.8) + labs(x = "Difference between imputed and true value") +  
      scale_x_continuous(limits = c(-15, 15), breaks = seq(-15,  
        15, 2)) + scale_y_continuous(expand = c(0, 0)) +  
      theme_classic() + theme(legend.title = element_blank(),  
        legend.position = "none", axis.text.y = element_blank(),  
        axis.title.y = element_blank(), axis.ticks.y = element_blank(),  
        axis.line.y = element_blank(), axis.text.x = element_text(size = 12),  
        axis.title.x = element_text(size = 15), plot.margin = margin(10,  
          10, 10, 10))  
  }
```



So, replacing every NA with a global mean is not acceptable. The next method we might consider is to estimate a local mean from the two nearest known values.

- **Local mean substitution:** Local mean substitution can already be achieved with the `approx()` function in the `base {stats}` package; however, this does not work nicely if we want to code using pipes. As an alternative, we can use the `na_interpolation()` function from the package `{imputeTS}`.

Here's what this looks like:

```
# Use the imputeTS package
imputeTS::na_interpolation(x = c(1, NA, 5), method = "constant",
  f = 0.5)
```

```
## [1] 1 3 5
```

```
## We need to specify the 'f' argument at 0.5 ## to weight
## each point equally.
```

Basically, the **constant** method allows you to treat the chunks of consecutive NAs equally. Therefore, they are assigned the same value. So, when you multiple chunks of NAs, only the nearest values are considered for imputation. For example,

```
imputeTS::na_interpolation(x = c(1, NA, NA, 5), method = "constant",
  f = 0.5)
```

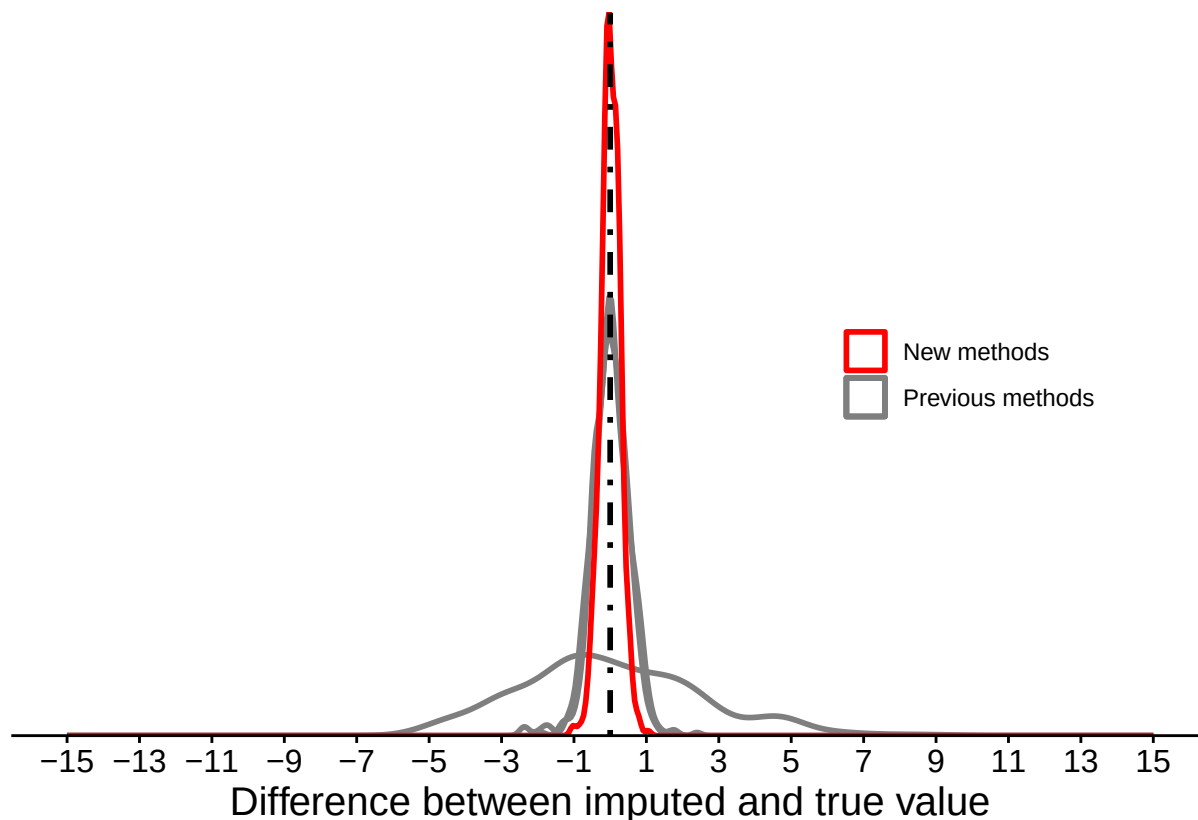
```
## [1] 1 3 3 5
```

```
imputeTS::na_interpolation(x = c(10, 1, NA, NA, 5, 1), method = "constant",
  f = 0.5)
```

```
## [1] 10  1  3  3  5  1
```

Now, let's apply this technique to our data:

```
### Replace missing values using
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_localmean = imputeTS::na_interpolation(x = PM2.5_missing,
    method = "constant", f = 0.5))
```



By using the two closest points to impute a local mean we can substantial improvement upon the method of global mean substitution; however, in many cases we can also gain information from other data points, not just the nearest neighbours. Of course, as we move further from the NA we will gain less and less information, so we can weight the information based on the distance from our missing data. This leads us to the **weighted moving average** method that we can employ with the `na_ma()` function from `{imputeTS}`.

- **Weighted Moving Average** : `na_ma()` has two key arguments that you need to understand:
- **k**: Size of the window over which to calculate the moving average. This is the number of time steps to either side of the NA that will be used. So, for an NA at position i and $k = 2$, the weighted moving averaging would use observations at $i - 2$, $i - 1$, $i + 1$ and $i + 2$.
- **weighting**: The method used to weight data. This can include:
 - **simple**: All values are equally weighted.
 - **linear**: Weight decreases linearly with distance from i . If x is the distance from i , then the value has a weight of $1/(1 + x)$, such that the nearest values have weight of $1/2$, $1/3$, $1/4$ etc.

- **exponential**: Weight decreases exponentially. If x is the distance from i , then value has a weight $1/2^x$ (default).

If $k=1$, we will get the same result as `na_interpolation()`. Let us see some examples:

```
imputeTS::na_ma(x = c(4, 3, 1, NA, 3, 4, 10), k = 1)
```

```
## [1] 4 3 1 2 3 4 10
```

So, we observed that we are getting the same output as `na_interpolation()`. If we expand the window (e.g. $k=3$), we include more information. Our result is no longer the same as using `na_interpolation()`.

```
imputeTS::na_ma(x = c(4, 3, 1, NA, 3, 4, 10), k = 3, weighting = "simple")
```

```
## [1] 4.000000 3.000000 1.000000 4.166667 3.000000
## [6] 4.000000 10.000000
```

Using equal weights for all points is not ideal, so we can try using alternative weighting methods. In our simple example, using linear or exponential weighting minimizes the influence of large values further away from NA.

```
imputeTS::na_ma(x = c(4, 3, 1, NA, 3, 4, 10), k = 3, weighting = "linear")
```

```
## [1] 4.000000 3.000000 1.000000 3.615385 3.000000
## [6] 4.000000 10.000000
```

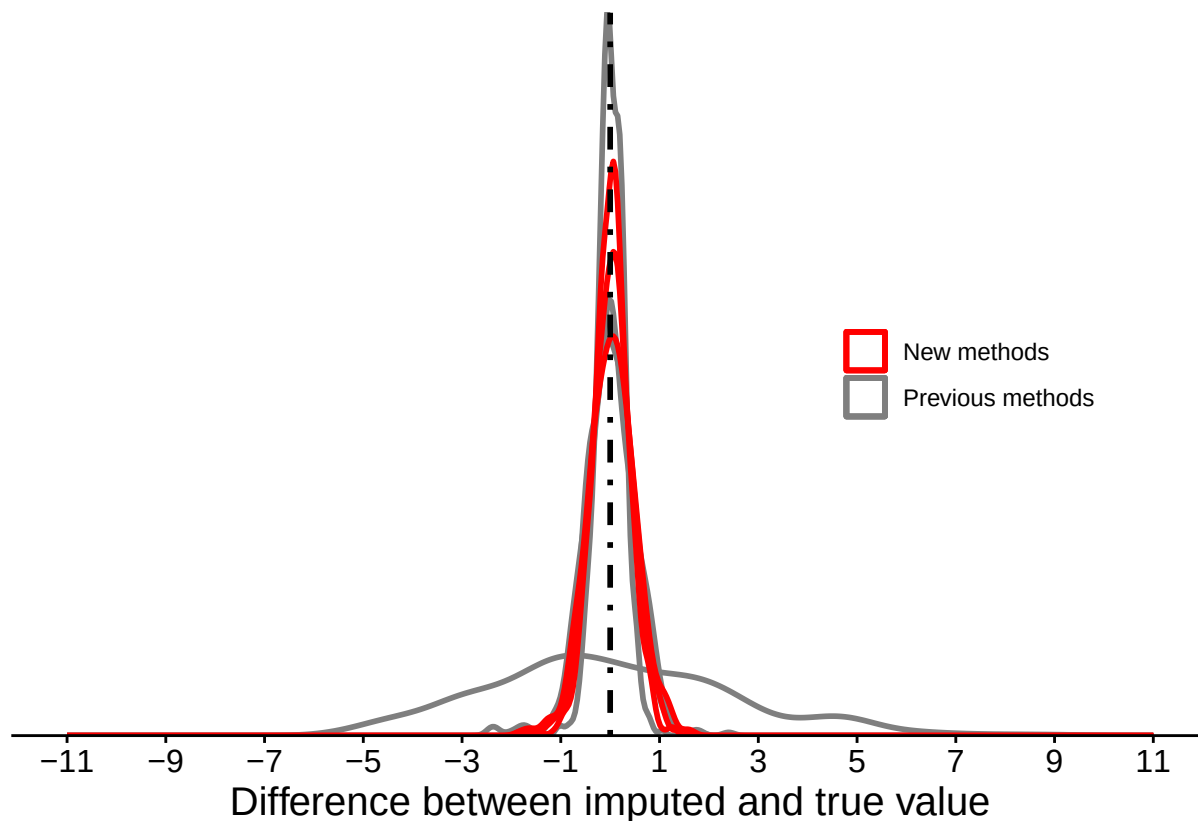
```
imputeTS::na_ma(x = c(4, 3, 1, NA, 3, 4, 10), k = 3, weighting = "exponential")
```

```
## [1] 4.000000 3.000000 1.000000 3.142857 3.000000
## [6] 4.000000 10.000000
```

To compare the benefit of different weighting methods we will use all three. For now, we just use default window size $k = 4$ (i.e. using 8 values in total towards mean).

```
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_mas = imputeTS::na_ma(x = PM2.5_missing, weighting = "simple"),
         PM2.5_mal = imputeTS::na_ma(x = PM2.5_missing, weighting = "linear"),
         PM2.5_mae = imputeTS::na_ma(x = PM2.5_missing, weighting = "exponential"))
```

Now we can compare all our substitution methods: forward fill, back fill, global, local, moving average (equal weight), moving average (linear weight), moving average (exponential weight). All methods, other than global mean, seem to perform reasonably well.



- Lin-

ear Imputation : Linear imputation is one of the more common alternatives. Like our basic local mean substitution, linear imputation uses information from the two nearest known points, but instead of taking the mean of these two points it imputes the missing value(s) assuming there is a linear trend in our variable over time.

To do this, we are going back to the `na_interpolation()` function, but this time we use the argument `option='linear'`. If we have a single NA value, linear imputation and local mean substitution will produce the same result.

```
# Assume a linear relationship of x over time,
imputeTS::na_interpolation(x = c(1, NA, NA, 3), option = "linear")
```

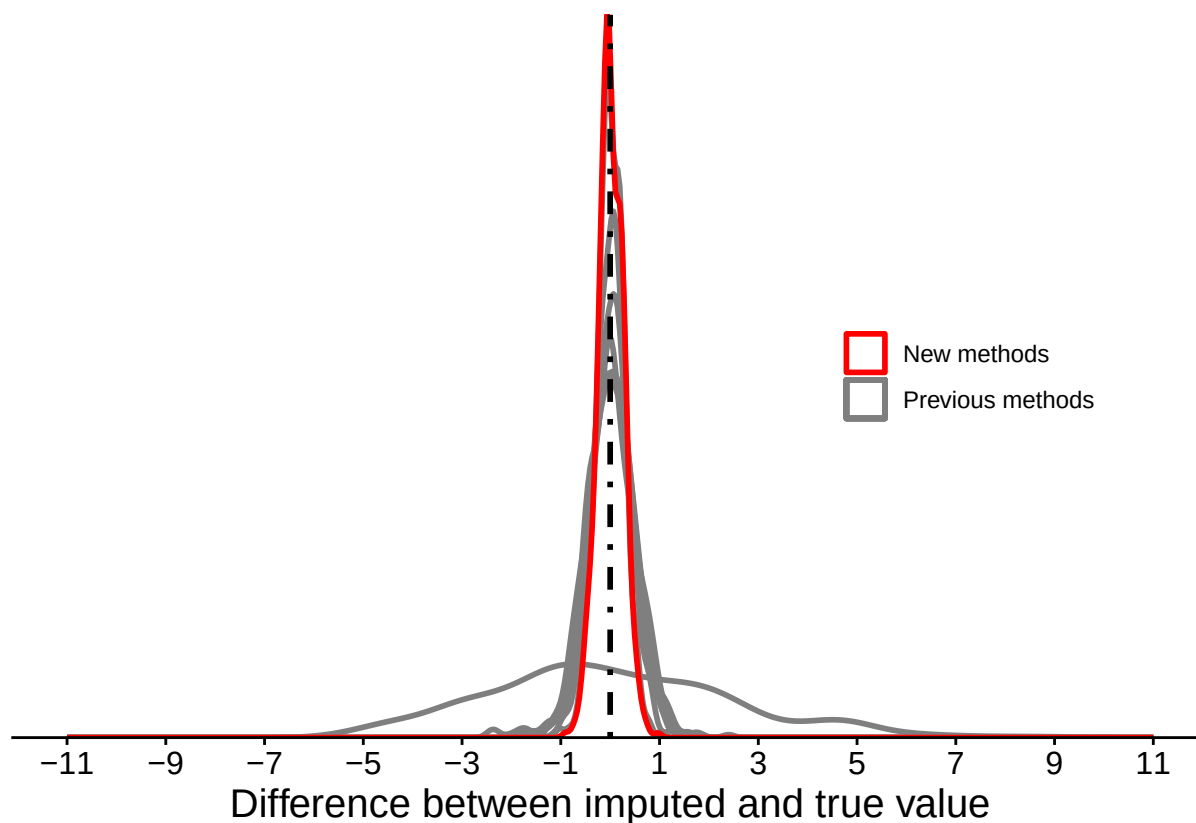
```
## [1] 1.000000 1.666667 2.333333 3.000000
```

```
# Take the mean of the two closest points
imputeTS::na_interpolation(x = c(1, NA, NA, 3), method = "constant",
  f = 0.5)
```

```
## [1] 1 2 2 3
```

If you observe the difference in the outputs between these functions are that unlike local mean substitution, chunks of NAs are given different values. Previously based on our `ggplot_na_gapsize`, we observed that most of the NAs in our data are non-consecutive, i.e., they are isolated from each other. So our linear imputation method will be very similar to local mean substitution in this case.

```
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_linearinterp = imputeTS::na_interpolation(x = PM2.5_missing,
    option = "linear"))
```



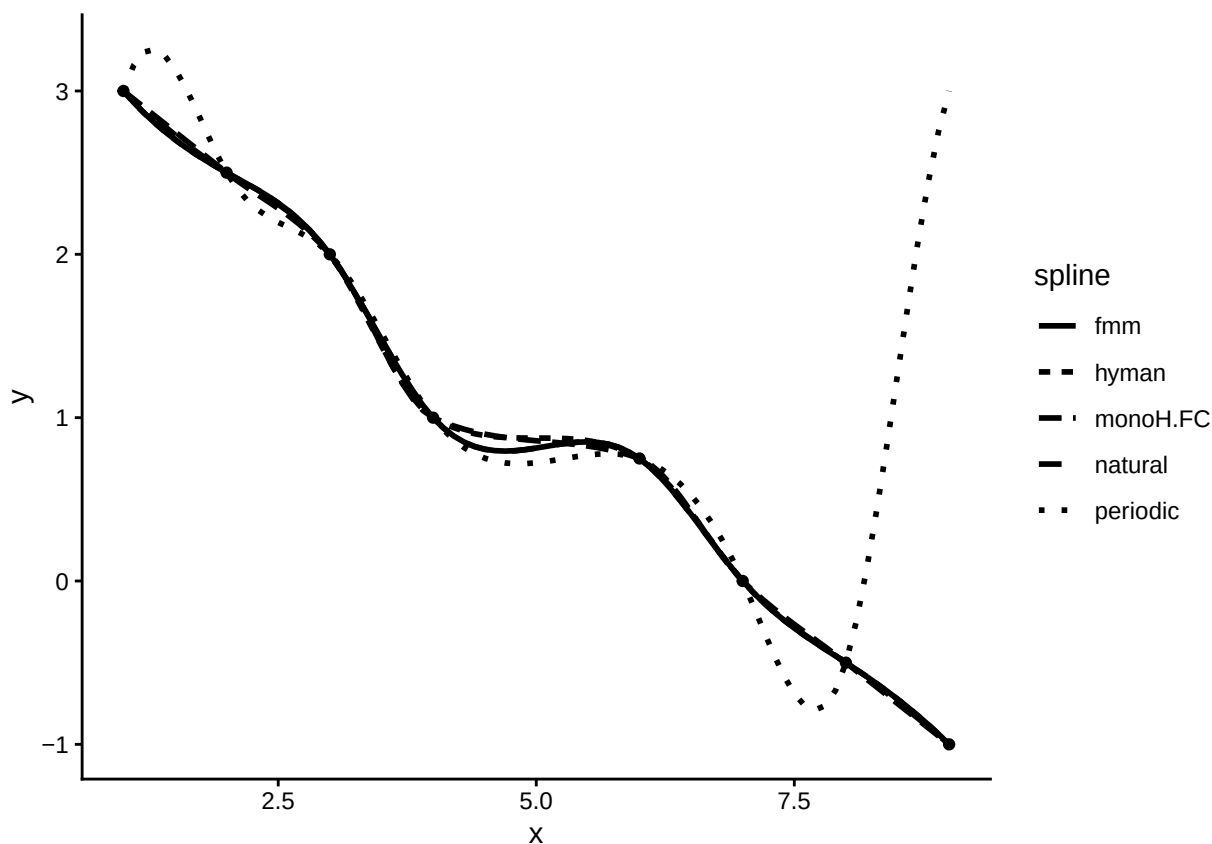
Spline Interpolation : If we want to relax the assumption of linear change over time, we can instead use spline interpolation. Spline interpolation, as the name suggests, imputes missing values by fitting a spline through nearby known points. Unlike linear interpolation, spline interpolation uses more data either side of missing value, making it potentially more powerful.

To achieve spline interpolation we still use the `na_interpolation()` function, but now specify `option = "spline"` instead of `"linear"`.

```
# Fit spline through known data to estimate NA
imputeTS::na_interpolation(x = c(3, 2.5, 2, 1, NA, 0.75, 0, -0.5,
  -1), option = "spline")
```

```
## [1] 3.0000000 2.5000000 2.0000000 1.0000000 0.8142007
## [6] 0.7500000 0.0000000 -0.5000000 -1.0000000
```

There are a range of different spline functions that might have different success. We can pick the spline method used with the `'method'` argument. There are **5 types** of splines available, such as, `fmm`, `periodic`, `natural`, `monoH.FC` and `hyman`. We can see the different spline fits below.



For our PM 2.5 concentration data, we focus on three methods only ('fmm', 'periodic', and 'natural'). The two other available methods ('monoH' and 'hyman') assume monotonic data (i.e. data should either never increase or decrease) and so are not appropriate for air quality data.

```
## Fit all different splines that we can compare
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_fmm = imputeTS::na_interpolation(x = PM2.5_missing,
    option = "spline", method = "fmm"), PM2.5_periodic = imputeTS::na_interpolation(x = PM2.5_missing,
    option = "spline", method = "periodic"), PM2.5_natural = imputeTS::na_interpolation(x = PM2.5_missing,
    option = "spline", method = "natural"))
```

- **Stineman imputation** : The final method we can try is the Stineman imputation method. The algorithm of **Stineman (1980)** is designed specifically for data imputation. We won't go into the algorithm details. But, you can find out more about the algorithm [here](#). One advantage of this method is it avoids generating any 'wild' points.

```
imputeTS::na_interpolation(x = c(3, 2.5, 2, 1, NA, 0.75, 0, -0.5,
  -1), option = "stine")
```

```
## [1] 3.000 2.500 2.000 1.000 0.875 0.750 0.000 -0.500
## [9] -1.000
```

```
df_filter_tibble <- df_filter_tibble %>%
  mutate(PM2.5_stine = imputeTS::na_interpolation(x = PM2.5_missing,
    option = "stine"))
```

Until now we've used a graphical comparison of real and imputed PM 2.5 concentration data to compare methods. This is good enough to spot bad methods (like global mean substitution), but it doesn't allow for a quantitative

comparison of methods. For a more refined comparison we can use metrics such as **mean squared error (MSE)** and **root mean squared error (RMSE)**. These two metrics are slightly different, but in both cases we want to minimize error. RMSE benefits from being in the same units as our variable (μ/m^3 in our case). MSE is not in true units, but is more sensitive to outliers.

```
validation_stats <- df_filter_tibble %>%
  as_tibble() %>%
  ## Filter only those cases where data are missing
  filter(is.na(PM2.5_missing)) %>%
  tidyr::pivot_longer(cols = PM2.5_ffill:PM2.5_stine, values_to = "imputed",
    names_to = "method") %>%
  mutate(sqdiff = (imputed - PM2.5)^2) %>%
  group_by(method) %>%
  summarise(MSE = sum(sqdiff)/n(), RMSE = sqrt(MSE)) %>%
  arrange(RMSE)

full_names <- data.frame(method = c("PM2.5_stine", "PM2.5_linearinterp",
  "PM2.5_localmean", "PM2.5_periodic", "PM2.5_fmm", "PM2.5_natural",
  "PM2.5_mae", "PM2.5_mal", "PM2.5_mas", "PM2.5_meansub", "PM2.5_bfill",
  "PM2.5_ffill"), name = c("Stineman", "Linear imputation",
  "Local mean", "Spline (periodic)", "Spline (fmm)", "Spline (Natural)",
  "Weighted mean (exponential)", "Weighted mean (linear)",
  "Weighted mean (Equal weight)", "Global mean", "Next Observation Carried Backward",
  "Last Observation Carried Forward"))

validation_stats %>%
  left_join(full_names, by = "method") %>%
  select(name, MSE, RMSE)
```

```
## # A tibble: 12 x 3
##   name                                MSE  RMSE
##   <chr>                            <dbl> <dbl>
## 1 Spline (periodic)                 0.0536 0.232
## 2 Spline (Natural)                 0.0536 0.232
## 3 Spline (fmm)                    0.0536 0.232
## 4 Stineman                        0.0607 0.246
## 5 Linear imputation               0.0656 0.256
## 6 Local mean                      0.0767 0.277
## 7 Weighted mean (exponential)      0.139  0.372
## 8 Weighted mean (linear)           0.183  0.428
## 9 Weighted mean (Equal weight)     0.243  0.493
## 10 Next Observation Carried Backward 0.244  0.494
## 11 Last Observation Carried Forward 0.280  0.529
## 12 Global mean                     6.24   2.50
```

As expected, global mean substitution is noticeably worse than all other methods. It's particularly bad in terms of MSE, as it can produce very large outliers. At the other end, all the Spline interpolation method comes out on top but it is almost indistinguishable from results with much simpler linear imputation or local mean substitution. For our air quality data on PM 2.5 concentration, any of these methods would likely be acceptable.